```python
# simulate_game_engine.py
import numpy as np
import pandas as pd
import pickle
import os
from collections import defaultdict
from dataclasses import dataclass, field
import sqlite3


# SQLite3 Data Loader
def load_team_data_from_db(db_path: str, team_names: list) -> pd.DataFrame:
    conn = sqlite3.connect(db_path)
    placeholders = ','.join(['?'] * len(team_names))
    query = f"SELECT * FROM bracket_data" if not team_names else f"SELECT * FROM bracket_data WHERE Team IN ({placeholders})"
    df = pd.read_sql_query(query, conn, params=team_names if team_names else None)
    conn.close()
    return df


db_path = os.path.join(os.path.dirname(__file__), "data", "bracket.db")
team_names = []


def get_models_and_data(db_path: str, team_names: list):
    df = load_team_data_from_db(db_path, team_names)

    model_path_xgb = os.path.join(os.path.dirname(__file__), "models", "xgb_model.pkl")
    with open(model_path_xgb, "rb") as f:
        XGB_model = pickle.load(f)

    expected_features = XGB_model.get_booster().feature_names
    if expected_features is None:
```

```
        drop_cols = {'Year', 'Score', 'Team', 'Result', 'Champion'}
        expected_features = [c for c in df.columns if c not in drop_cols]
    return df, XGB_model, expected_features


# Initialize both expected feature
EXPECTED_FEATURES = None

# Constants
ROUND_NOISE_FACTORS = {64: 1.15, 32: 1.00, 16: 0.90, 8: 0.85, 4: 0.80, 2: 0.75}
ROUND_MEAN_ACTUAL = {64: 74.5, 32: 73.0, 16: 71.5, 8: 70.5, 4: 69.5, 2: 66.0}
seed_win_baseline = {
    "R64": {
        1: 0.987, 2: 0.929, 3: 0.853, 4: 0.788, 5: 0.647,
        6: 0.609, 7: 0.615, 8: 0.481, 9: 0.519, 10: 0.385,
        11: 0.391, 12: 0.353, 13: 0.212, 14: 0.147, 15: 0.071, 16: 0.013,
    },
    "R32": {
        1: 0.846, 2: 0.635, 3: 0.526, 4: 0.474, 5: 0.346,
        6: 0.288, 7: 0.186, 8: 0.103, 9: 0.051, 10: 0.154,
        11: 0.173, 12: 0.141, 13: 0.038, 14: 0.013, 15: 0.026,
    },
    "Sweet 16": {
        1: 0.660, 2: 0.442, 3: 0.256, 4: 0.160, 5: 0.077,
        6: 0.109, 7: 0.064, 8: 0.058, 9: 0.032, 10: 0.058,
        11: 0.064, 12: 0.013, 13: 0.000, 14: 0.013, 15: 0.006,
    },
    "Elite 8": {
```

```
        1: 0.397, 2: 0.205, 3: 0.109, 4: 0.096, 5: 0.058,
        6: 0.019, 7: 0.019, 8: 0.038, 9: 0.013, 10: 0.006,
        11: 0.038, 12: 0.000, 13: 0.000, 14: 0.000, 15: 0.000,
    },
    "Final Four": {
        1: 0.250, 2: 0.083, 3: 0.071, 4: 0.026, 5: 0.026,
        6: 0.013, 7: 0.006, 8: 0.026, 9: 0.000, 10: 0.000,
        11: 0.000, 12: 0.000, 13: 0.000, 14: 0.000, 15: 0.000,
    },
    "Championship": {
        1: 0.160, 2: 0.032, 3: 0.026, 4: 0.013, 5: 0.000,
        6: 0.006, 7: 0.006, 8: 0.006, 9: 0.000, 10: 0.000,
        11: 0.000, 12: 0.000, 13: 0.000, 14: 0.000, 15: 0.000,
    },
}

score_by_round = defaultdict(list)


def init_score_by_round():
    """Reset per-round simulation starts scoring so each fresh."""
    global score_by_round
    score_by_round = defaultdict(list)


# Team class
@dataclass
class Team:
    name: str
    seed: int
    stats: dict = field(default_factory=dict)
```

```


# Enrichment functions
def get_team_stats(team_name: str, df: pd.DataFrame) -> pd.Series:
    team_rows = df[df["Team"] == team_name]
    if team_rows.empty:
        zero_stats = {feature: 0.0 for feature in EXPECTED_FEATURES}
        zero_stats["Team"] = team_name
        zero_stats["Current Round"] = 64.0
        zero_stats["Seed"] = 16.0
        zero_stats["Opponent Seed"] = 1.0
        return pd.Series(zero_stats)
    else:
        if "Current Round" in df.columns:
            r64 = team_rows[team_rows["Current Round"] == 64]
            result = r64.iloc[0] if not r64.empty else team_rows.iloc[0]
        else:
            result = team_rows.iloc[0]

        for col in ['Team', 'Year', 'Current Round']:
            if col in result:
                result = result.drop(col)
        return result


def enrich_team(team, db_path, expected_features):
    conn = sqlite3.connect(db_path)
    row = pd.read_sql_query(
        "SELECT * FROM bracket_data WHERE Team = ?",
        conn,
        params=(team.name,)
    )
    conn.close()

    if row.empty:
        team.stats = {f: 0.0 for f in
```

```
expected_features}
    else:
        stats_row = row.iloc[0].to_dict()
        for col in ['Team', 'Year', 'Current Round']:
            stats_row.pop(col, None)
        team.stats = stats_row

    return team


def simulate_game(team1, seed1, team2, seed2,
                  db_path, model, expected_features,
                  noise_std=0.0,
                  consistency_multiplier=1.0,
                  blend_weight=0.0,
                  normalize_by_round=False,
                  current_round=None):
    team1 = enrich_team(team1, db_path, expected_features)
    team2 = enrich_team(team2, db_path, expected_features)

    # Assign opponent seeds
    team1.stats['Opponent Seed'] = seed2
    team2.stats['Opponent Seed'] = seed1

    # Prepare inputs
    feats1 = {f: team1.stats.get(f, 0.0) for f in expected_features}
    feats2 = {f: team2.stats.get(f, 0.0) for f in expected_features}
    X1 = pd.DataFrame([feats1], columns=expected_features)
    X2 = pd.DataFrame([feats2], columns=expected_features)

    # Predict
    s1 = float(model.predict(X1)[0])
    s2 = float(model.predict(X2)[0])

```

```
    if current_round is not None and normalize_by_round:
        round_scalers = {
            "First Four": 0.65,
            "Round of 64": 1.0,
            "Round of 32": 0.9,
            "Sweet Sixteen": 0.8,
            "Elite Eight": 0.75,
            "Final Four": 0.7,
            "Championship": 0.65
        }
        scaler = round_scalers.get(current_round, 1.0)
        s1 *= scaler
        s2 *= scaler

    # Seed consistency bias
    if seed1 < seed2:  # team1 is better
        s1 += consistency_multiplier * abs(seed2 - seed1)
        s2 -= consistency_multiplier * abs(seed2 - seed1)
    elif seed2 < seed1:  # team2 is better
        s1 -= consistency_multiplier * abs(seed2 - seed1)
        s2 += consistency_multiplier * abs(seed2 - seed1)

    # Blend with history
    if blend_weight > 0:
        hist_margin = (seed2 - seed1) * 1.5
        s1 = s1 * (1 - blend_weight) + (s1 + hist_margin) * blend_weight
        s2 = s2 * (1 - blend_weight) + (s2 - hist_margin) * blend_weight

    # Chaos
    noise_scale = (abs(s1 - s2) / 5.0)
    s1 += np.random.normal(0, noise_std * noise_scale)
    s2 += np.random.normal(0, noise_std *
```

```
noise_scale)

    # Normalize by round
    if normalize_by_round:
        mean_target = ROUND_MEAN_ACTUAL.get(64, (s1 + s2) / 2)
        scale_factor = mean_target / ((s1 + s2) / 2)
        s1 *= scale_factor
        s2 *= scale_factor

    winner = team1 if s1 > s2 else team2

    # Ensure scores are reasonable values (no negative scores)
    s1 = max(40, s1)
    s2 = max(40, s2)

    return {"winner": winner, "t1_score": s1, "t2_score": s2}
```